

Complemento del Plan de Pruebas de TécnicoYa Huancayo

Control documental

Versión 1.1 propuesta para revisión académica. Fecha de corte: 22 de septiembre de 2026. Complementa la versión 1.0 del 7 de septiembre y conserva sus 31 páginas, los 33 RF, los 14 RNF y las decisiones pendientes. No sustituye firmas ni aprobaciones.

Finalidad

Vincular Scrum, diseño técnico, backend Laravel, PostgreSQL, matching FastAPI, frontend React, pruebas y despliegues mediante evidencias identificables por iteración. El archivo de ejemplos es la referencia de organización y de trazabilidad; sus casos no ejecutados no pasan a aprobados por copiarse al expediente.

Cómo leer el plan consolidado

El plan consolidado comienza con este complemento operativo y continúa con el PDF original íntegro. La numeración del original se conserva dentro de sus páginas. Las propuestas de este complemento requieren revisión del equipo; los criterios originales siguen vigentes salvo cambio aprobado y registrado.

Estado comprobable

Se inspeccionaron el código de pruebas, las reglas, las migraciones y la configuración de despliegue. La evidencia nueva se guarda en evidencias/revision-2026-09-22. Los resultados históricos mencionados en conversaciones no sustituyen logs reproducibles. No existe en este expediente evidencia de despliegue productivo, ensayo de carga, certificación ISO o aceptación firmada.

Scrum y entregas por iteración

Marco de trabajo

Scrum organiza el desarrollo del producto; no es una metodología de base de datos. PostgreSQL es el gestor relacional. Se propone un Sprint de una semana, sujeto al calendario académico. El Product Owner prioriza y aclara criterios, el Scrum Master facilita el proceso y Developers construyen y verifican el incremento. Las asignaciones nominales siguen pendientes; no se equiparan automáticamente con la RACI del plan.

Secuencia dentro de cada Sprint

Seleccionar historias y objetivo; precisar datos y permisos; revisar diseño y contratos API; implementar migraciones, backend y frontend; ejecutar pruebas; registrar defectos y retest; preparar un incremento desplegable; desplegar en QA cuando el entorno y criterios estén listos; revisar resultados y hacer retrospectiva. Diseño, datos y pruebas se refinan en cada iteración, no se aplazan al final del proyecto.

Eventos y evidencias

Planning: objetivo y Sprint Backlog. Daily Scrum: seguimiento al objetivo, con impedimentos relevantes. Review: demostración del incremento y feedback. Retrospectiva: mejora concreta con responsable. Los registros históricos no se reconstruyen como reuniones realizadas si no existen actas.

Despliegue

Cada iteración planificada incluye un despliegue a QA como objetivo del proyecto. Scrum exige un incremento utilizable conforme a la Definition of Done; no obliga a desplegar a producción en cada Sprint. Si falla el despliegue, se registra bloqueado con causa, no realizado. La liberación no necesita esperar a la Sprint Review.

Definition of Done propuesta

Criterios verificables cumplidos; revisión de código; pruebas relevantes y regresión aprobadas; cambios de datos versionados; permisos negativos comprobados; documentación actualizada; defectos críticos resueltos; build identificado; evidencia de instalación y smoke en QA para declarar el despliegue completado.

Diseño técnico y estrategia de datos

Arquitectura observada

React en frontend/web consume Laravel en backend/api. Laravel autentica, autoriza, conserva el ciclo de solicitudes y consulta FastAPI en python/api para ordenar candidatos. Si Python falla, utiliza matching local PHP. PostgreSQL es la fuente de verdad del modo conectado. El algoritmo JavaScript de demo no reemplaza al backend ni demuestra persistencia real.

Base de datos

Las migraciones de backend/api/database/migrations son el mecanismo de instalación. database contiene el esquema de referencia. Deben compararse ambos antes de cerrar una iteración. Usar datos sintéticos; no incorporar credenciales ni información personal real en evidencias. Probar claves, relaciones, unicidad, transacciones y restauración en una instancia PostgreSQL de QA.

Interacciones técnicas

Cliente registra falla; Laravel valida y persiste; matching filtra y ordena; se generan hasta tres ofertas; técnico acepta; cliente confirma; técnico inicia y finaliza; cliente califica. Revisar la matriz actor por transición y propiedad del recurso en API, no solo ocultar botones.

Reglas centralizadas

domain/rules.json fija respuesta de 10 minutos, máximo 3 notificados, ventana de 24 horas y calificación dentro de 48 horas. Pesos: cercanía 35%, tarifa 20%, historial 30%, experiencia 15%. Desempate por calificación y orden estable adicional por identificador. Cercanía usa cobertura de zona; no representa distancia GPS.

Brechas que requieren casos propios

No considerar cubiertos por el flujo básico: fotografías, distancia geográfica, agenda y reprogramación, atención por varios técnicos, apelaciones, métodos de pago y notificación externa con reintentos. Las pruebas Laravel inspeccionadas usan SQLite en memoria y un doble HTTP de Python; no demuestran concurrencia PostgreSQL ni integración de red Laravel a FastAPI.

Pruebas y trazabilidad del código

Frontend

frontend/web/src/domain/marketplace.test.js: reglas y ciclo demo. frontend/web/src/forms.test.jsx: formulario. frontend/web/src/VideoExperience.test.jsx: introducción del video. frontend/web/src/scroll/scroll.test.jsx: tiempos, caché y movimiento reducido. Ejecutar npm test y npm run build desde frontend/web. Estas comprobaciones de interfaz no son por sí solas aceptación de todos los RF.

Backend

backend/api/tests/Feature/MarketplaceTest.php contiene seis métodos: registro y sesión revocable, ciclo autenticado, plazos y tres ofertas, denegación de alta de administrador y acciones ajenas, rechazo de calificación vencida y fallback de matching. Ejecutar php artisan test desde backend/api. Estado en este corte: código inspeccionado; ejecución nueva no adjunta.

Python

python/api/test_matching.py contiene tres pruebas: fixture compartida, validaciones y resultado vacío, desempate por calificación. Ejecutar python -m pytest -q desde python/api con dependencias instaladas. Estado en este corte: código inspeccionado; ejecución nueva no adjunta.

Relación provisional con los casos

Matching y fallback aportan evidencia parcial a CT-06/08/09. Los límites y ofertas aportan a CT-13/14/16/17. El ciclo aporta a CT-18/19/22/23/25. La autenticación y permisos aportan a CT-28/33. Esta relación no implica aprobación completa: cada CT incluye condiciones adicionales del plan y debe ejecutar sus variantes.

Registro requerido

Relacionar RF o RNF, condición, CT o CNF, procedimiento, datos, commit, entorno, resultado esperado, resultado observado, log, defecto, retest y responsable. La matriz CSV conserva los 33 RF y 14 RNF del ejemplo; el estado global permanece pendiente de verificación integral. Un test aprobado no equivale a cobertura del 70% sin definir denominador y medirla.

Aplicación de normas y criterios de calidad

Gestión de calidad

ISO 9001 se utiliza como referencia de control de versiones, responsables, riesgos, revisión y mejora. El expediente conserva la referencia 2015 de los ejemplos; debe formalizarse la edición académica aplicable y revisar cambios de edición antes de evaluar conformidad. No se declara certificación ni cumplimiento de todas las cláusulas.

Calidad del producto

Proponer ISO/IEC 25010:2023 como línea base del producto, pendiente de aprobación. El ejemplo mantiene ocho características de producto y cinco de calidad en uso de 2011: no tratar ese esquema como cobertura completa de 2023. La calidad en uso se aborda separadamente; mantener la decisión de edición visible en decisiones.csv.

Proceso y documentación de pruebas

ISO/IEC/IEEE 29119-2:2021 orienta planificación, seguimiento, ejecución y cierre; 29119-3:2021 orienta documentación. Conservar plan, especificaciones, procedimientos, logs, incidentes y cierre. El ejemplo propone 29119-4:2021 para técnicas: aplicar particiones, valores límite, decisiones y transiciones de estado según los casos. Esto es alineación metodológica, no auditoría normativa completa.

Criterios que no deben relajarse

Conservar los RNF del ejemplo: matching de 3 s, 200 usuarios concurrentes, SUS de 70, disponibilidad mensual del 99% y cobertura automatizada del 70% con métrica aún por definir. Resolver el conflicto de 3 s frente a 5 s antes del ensayo; definir carga, percentil, duración y dataset. Un build local no valida disponibilidad mensual, TLS, experiencia de usuarios o carga.

Fuentes normativas consultadas

Catálogos oficiales: <https://www.iso.org/standard/78176.html> ; <https://www.iso.org/standard/79428.html> ; <https://www.iso.org/standard/79429.html> . Referencia ISO 9001 del ejemplo: <https://committee.iso.org/sites/tc176sc2/home/projects/published/iso-9001-2015.html> . Scrum Guide: <https://scrumguides.org/scrum-guide.html> . Consulta: 22/09/2026. No se dispone del texto íntegro licenciado de las normas para auditar cláusula por cláusula.

Plan de iteraciones y despliegue

Plan propuesto y no historial ejecutado

S01: acceso y perfiles, autenticación, migraciones iniciales, UI, permisos y smoke QA. S02: solicitud y matching integrado, datos sintéticos y caída de Python. S03: ofertas, aceptación y ejecución con plazos y concurrencia PostgreSQL. S04: calificación, administración, cobertura y regresión. Las fechas, responsables y capacidad se acuerdan antes del Planning. El código ya existente es punto de partida, no evidencia de que estos sprints ocurrieron.

Ficha por Sprint

Cada carpeta S01 a S04 contiene objetivo propuesto, alcance, tareas de datos/backend/frontend, pruebas, despliegue, review y retrospectiva, todos con estado pendiente salvo evidencia adjunta. No inventar commits, URLs, firmas, fechas de reunión ni actas. Registrar alcance diferido y trasladarlo al Product Backlog.

Procedimiento de QA

Identificar commit y artefactos; preparar variables fuera del repositorio; levantar PostgreSQL y servicios en entorno aislado; ejecutar migraciones sobre copia controlada; comprobar health, login y solicitud sintética; revisar logs y permisos; guardar resultado y URL. `infrastructure/docker-compose.yml` es una configuración existente, no un comprobante de ejecución. `.gitlab-ci.yml` referencia un pipeline externo que debe verificarse en GitLab.

Rollback y cierre

Antes de migrar, registrar respaldo y procedimiento de recuperación; después, comparar integridad. No ejecutar rollback destructivo sobre datos reales como prueba. Ante fallo, conservar logs, abrir defecto y registrar la entrega como bloqueada. Cerrar solo con evidencia, defectos y riesgos conocidos; aprobación académica pendiente.

Actualización del plan

Al cerrar cada ciclo: actualizar matriz RF/CNF, incidentes y decisiones; adjuntar logs y capturas sin secretos; generar informe de Sprint; actualizar versión del plan e índice documental. La evidencia anterior se conserva con su build y entorno. No sobrescribir un resultado fallido con el retest.

Estado del arte y control del expediente

Investigación conservada

fuentes/estado-del-arte-original.pdf contiene el trabajo de investigación entregado.
trazabilidad/estado-del-arte.csv conserva las doce relaciones del archivo de ejemplos: asignación multicriterio, disponibilidad, distancia, experiencia, reputación, usabilidad y cobertura. No se presentan como artículos nuevos revisados ni se sustituyen referencias originales.

Conexión con el producto

Los antecedentes orientan hipótesis y casos: consistencia del ranking; exclusión de no disponibles; calidad de calificaciones; comprensión de tarifa; eficiencia de tareas y cobertura por zona. Su existencia no prueba que un algoritmo de la literatura esté implementado: TécnicoYa usa la fórmula documentada en domain/rules.json y el código de matching.

Organización

fuentes: originales y hashes. plan-de-pruebas: complemento y PDF consolidado. trazabilidad: RF, RNF, decisiones y antecedentes. sprints: fichas planificadas. diseno-tecnico: referencias de arquitectura y datos. evidencias: logs reales separados de plantillas. README.md: punto de entrada del expediente.

Criterio de evidencia

Realizado requiere artefacto y resultado verificable. Inspeccionado significa lectura del código, no ejecución. Propuesto significa trabajo planificado. Pendiente significa sin evidencia suficiente. Bloqueado exige causa registrada. La aceptación requiere criterios del plan y aprobación del responsable, nunca solo número de tests.

Siguientes decisiones

Formalizar roles Scrum y calendario; aprobar edición ISO y métricas de RNF; ejecutar integración real PostgreSQL y FastAPI; validar despliegue QA; ampliar casos de funcionalidades pendientes; revisar el expediente con el docente. Conservar los identificadores DP-01 a DP-18 del plan y registrar cambios con fecha y autor.